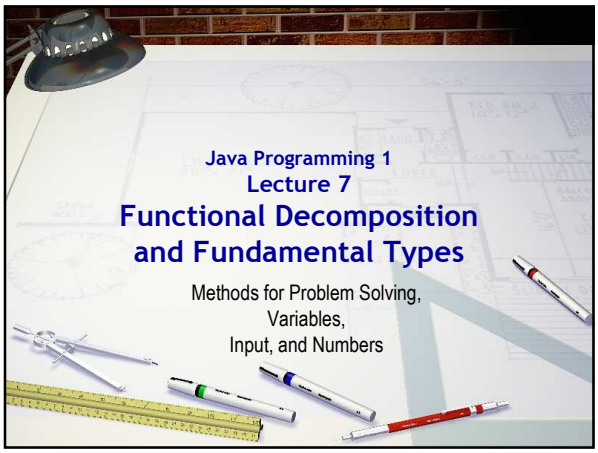


Express Yourself

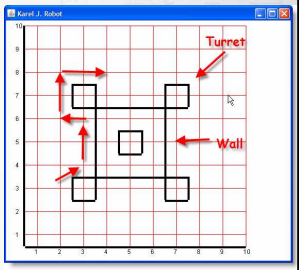
- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic07
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 7"
- **Exercise 1**: copy **PatrolTheCastle** project from Q: and open it in BlueJ. Create an object and call **task**().

A Visit From Karel

- Methods can be used for more than just setting values or returning information about an object.
  - Provide a powerful way to solve a problem
  - Put multiple steps into a single, simple method
  - Functional decomposition or divide and conquer
- Your last Karel problem was fairly long, a lot of steps
  - Let's use functional decomposition to complete a more complex job, in a simpler way

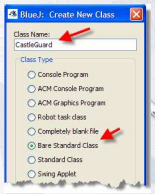
Karel Patrols the Castle

- Here's the castle and the route we want Karel to follow
- Think how many lines you'd have to write to patrol once around
  - Hard to understand
  - Hard to find your mistakes
- What if we had a robot that already knew how to patrol a castle?



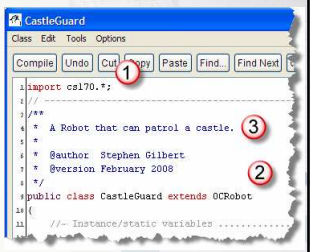
Our First CastleGuard

- Inheritance allows us to create a new specialized version of our **OCRobot** and add new commands
- **Exercise 2**: add a new Bare Standard Class to your project, and name it **CastleGuard**. Then, open the file in your editor.



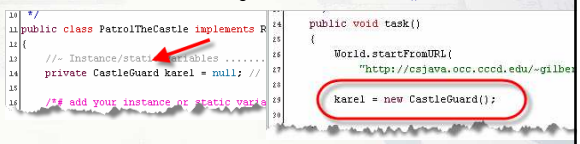
Setting Up CastleGuard

- Add these changes to **CastleGuard** and compile:
  - **import cs170.*;** on line 1
  - **extends OCRobot** on the header line
  - Add a comment and your name at the top of the class



Setting Up the Task

- Add these changes to **PatrolTheCastle**
  - Change the **OCRobot** variable to a **CastleGuard**
    - Ignore the "don't modify this" comment
  - Initialize karel using the **CastleGuard()** constructor



- **Exercise 3**: create a **PatrolTheCastle**, call **task()**

Adding turnRight

- The **OCRobot** can't **turnRight()** so you had a bunch of **turnLeft()** commands in your last homework.
- To add **turnRight()** to **CastleGuard**, add a method

```
public void turnRight()
{
   turnLeft();
   turnLeft();
   turnLeft();
}
```

Moving to the Start

- We can do the same thing to move Karel into the starting position, even though we'll only do it once

```
public void goToWork()
{
   move();
   move();
   move();
   turnRight();
   move();
   move();
   turnLeft();
}
```

Patrolling the Wall

- Patrolling a wall is very simple, just two **moves()**
- **Exercise 4**: add the **patrolWall()** method to the **CastleGuard** robot and compile.
- To use the new methods, just add this code to your **PatrolTheCastle** program. Then, create an instance of your program and call **task()**

```
karel.goToWork();
karel.patrolWall();
```

Patrolling a Turret

- Patrolling a turret is a little more complex. You need to use **turnLeft()**, **move()** and **turnRight()** to bring karel so he's ready to patrol the next section of wall.
- **Exercise 5**: write the **patrolTurret()** method and then add one line to your robot task class to call your method. Create an instance of your task class and call it's **task()** method to make sure Karel is lined up OK.
- **Exercise 6**: call **patrolWall()** and **patrolTurret()** until karel has completed his rounds. Turn him off.

Improving the Artist

- **Exercise 7**: Start BlueJ and create a new project named ic07. Import the file **Artist.java** from ic05.
  - Open the **Artist** class and show me a screen shot.
- How "easy to understand" is the code in Artist?
  - Can you tell by reading that it draws a house?

Literals and Magic Numbers

- When you call a method you pass information like this:

```
g.drawRect( 0, 0, 100, 100 );
```

  - The values 0 and 100 are called **literals** or **constants**
- Problems with literals
  - No meaning associated with number (magic number)
  - Making changes to code can be difficult
- Solution? Use **named values** called **variables**

Review: What are Variables?

- Programming languages store their data in **variables**

"Named, (usually changeable) storage area that holds a value"

- Like a "mailbox" or "bucket" that holds information

Review: What are Values?

- The data stored inside a variable is called its **value**
  - A variable can have different values as a program runs
- Different **kinds** of values we can put in a variable
  - We can store different numbers **or** text in a variable

```
PI = 3.14159;
myAge = 54;
myStreetNumber = "575";
```

  - **PI** holds a **real** number, **myAge** a **whole** number
  - **myStreetNumber** contains characters
    - Can't perform arithmetic on **myStreetNumber**

Review: What are data types?

- Different **kinds** of variables for different **kinds** of data
- Different **sizes** of containers for **same kind** of data

Review: Creating Variables

- You can define (or create) variables in two places:
  - Inside a method : called a **local variable**
  - Outside a method : called a **field** or **instance variable**
- You define a local variable using this syntax:

```
type name = initial-value;
```

  - Parts in gray are optional.
  - The parts in red are required

Review: Assignment

- Values are placed in variables using **assignment**

```
int num = 23;   // initializing assignment
num = 45;       // executable assignment
```

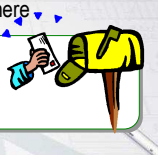
- Value can be from **input**, a **literal**, or an **expression**

```
num = readInt("Enter a whole number");
PI = 3.14159;
area = PI * radius * radius;
```

Review: How It Works

- Unlike a mailbox, a variable can hold **exactly one** value
  - Placing a value in a variable, (assignment), **replaces** the value previously stored there

```
myAge = 58;
kathysAge = MyAge;
kathysAge = 57;
```



  - **kathysAge** contains 58, **then** 57
  - **myAge** contains 58 for the length of the program
- We say that assignment performs a **destructive copy**

Assignment

- In Java, assignment is an operator, not statement
  - The job of the assignment operator is to:

Copy the value on its right into the variable on its left, and return the result copied as its value

- The variable on the left is changed when this happens
  - (Remember, this is a **destructive copy**)

Review: Kinds of Assignment

- The assignment operator is used in two situations
- 1) to give an initial value to a **new variable** at creation

```
int x = 5; // initialize i to 5
```

  - This is a **declaration**, **not** an executable statement
  - It can appear outside of any method
- 2) to copy a value into an **existing** variable

```
x = 23; // copy (store) the value 23 in location x
```

  - Executable statement; **must** appear inside a method

Review: Assignment or Equal

- Assignment is not "equality" (like algebra)

```
x = x + 1; // OK in Java, nonsense in algebra
```

- Means that the operand on the left **must** be a variable

```
13 = a + b; // OK in algebra, nonsense in Java
```

- Java uses a different operator to test equality

```
if ( a == 5 ) ... ; // Not ( a = 5 )
```

Defining Local Variables

- So far, we've only learned about a few types
  - Type **int** stands for whole numbers
  - Type **String** stands for a sequence of characters
- Here are some example declarations:

```
int x = 10;
int roofWidth = 300;
int roofHeight = 100;
int center = x + roofWidth / 2;
```

Artistic Variables

- Modify **Artist** to use these and other variables
  - Create variables to represent the position of the house
  - Create variables for the width and height of the house
  - Create variable for the width and height of the roof
    - Can these depend on the size of the house?
  - Create variables for the width and height of the door
  - Create variable for height and width of each window
- Is the modified program easier to understand?
- Modify some of the "base" variables. What happens?
- **Exercise 8**: Show me your code and a screen-shot of your running program.

Constants

- Named constants are "variables that don't vary"
  - Make code clearer by eliminating **magic numbers**
  - Create by adding modifier **final** in front of declaration
  - For instance variables, use **public static final**
  - Convention is to use **UPPER_CASE** for names

```
8  public class TempConverter
9  {
10    //-----------------------------------------------------------------
11    //  Computes the Fahrenheit equivalent of a specific Celsius
12    //  value using the formula F = (9/5)C + 32.
13    //-----------------------------------------------------------------
14    public static void main (String[] args)
15    {
16       final int BASE = 32;
17       final double CONVERSION_FACTOR = 9.0 / 5.0;
18
19       double fahrenheitTemp;
20       int celsiusTemp = 24;  // value to convert
21
22       fahrenheitTemp = celsiusTemp * CONVERSION_FACTOR + BASE;
```

Interactive Programs

- So far, our programs have been **static**, *not* dynamic
  - They work **exactly** the same each time they run
- We make our programs dynamic or interactive by having them process input supplied by the user
  - Input can be commands (mouse, menu or keyboard)
  - Input can also be data, which we'll store in variables
- The basic pattern for interactive programs:
  - Create a variable to hold the user's input
  - Ask the user to enter some data (called a **prompt**)
  - **Read** the data, convert it, and **store** it in the variable

The Scanner Class

- The **Scanner** class simplifies console input
  - Import the **java.util.Scanner** class
  - Create a **Scanner** variable using **System.in**
  - Call **nextLine()**, and store input in a **String** variable

```
8  import java.util.Scanner;
9
10 public class Echo
11 {
12    //-----------------------------------------------------------------
13    //  Reads a character string from the user and prints it.
14    //-----------------------------------------------------------------
15    public static void main (String[] args)
16    {
17       Scanner scan = new Scanner (System.in);
18
19       System.out.println ("Enter a line of text:");
20
21       String message = scan.nextLine();
22
23       System.out.println ("You entered: \"" + message + "\"");
24    }
25 }
26
```

Express Yourself

- **Exercise 9**: Create an interactive program that asks the user for their name, and then replies, "Hello <name>, I think I love you!". Use this pattern:
  - Create a BlueJ Console program
  - Make sure you import the **java.util.Scanner** class
  - Create a **Scanner** variable in your program
  - Create a **String** variable to hold the user's name
  - Ask the user to enter their name
  - Read the name using the **Scanner** object
  - Print the reply, using the name entered by the user

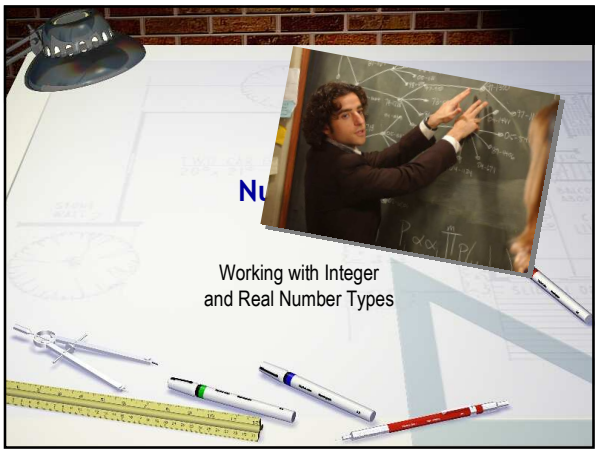
ACM Input

- ACM programs have their own input methods
  - Part of the **ConsoleProgram** class
- To read a line of text from the keyboard use **readLine()**

```
8   import acm.program.*;
9
10  public class ACMEcho extends ConsoleProgram
11  {
12     //-----------------------------------------------------------------
13     //  Reads a character string from the user and prints it.
14     //-----------------------------------------------------------------
15     public void run ()
16     {
17        String message = readLine("Enter a line of text: ");
18        println ("You entered: \"" + message + "\"");
19     }
20  }
21
```

Express Yourself

- **Exercise 10**: Create the same program, using the ACM ConsoleProgram class. Use this pattern:
  - Create a BlueJ ACM Console program
  - Create a **String** variable to hold the user's name
  - Read the user's name using the **readLine()** method
    - Don't need a separate prompt line. Just ask the user to enter their name when calling **readLine()**
  - Print the reply, using the name entered by the user

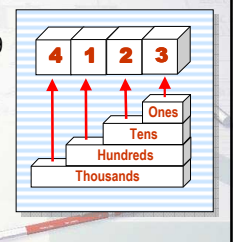


The Great Divide

- Java programs are composed of cooperating objects
  - You've used **Applet**, **Color**, and **Graphics**, etc.
  - Peer inside, and you'll find simpler components
- At the bottom - the **bit** : "atoms" of the computing world
- One step up are **primitive** or **fundamental** types
  - Similar to objects: have state and behavior
  - Value types rather than reference types
  - Unlike objects, can only hold a single, simple value
  - Manipulated by operators rather than methods
  - Behavior is built-in; it cannot be extended

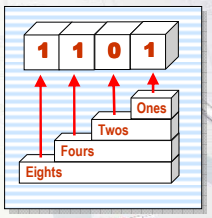
Decimal Numbers

- Internally, computers use **binary** to represent everything from whole numbers to real numbers to text to graphics
  - A binary number uses base 2 instead of base 10
- In **base 10**, or **decimal**:
  - We use ten digits, the digits 0 to 9
  - Each digit's value is based on its **positional value**
  - 4123 is 3 ones, 2 tens, 1 hundreds and four thousands



Binary Numbers

- In the base 2 or **binary** number system
  - We use only two digits 0 and 1
  - Instead of the tens, hundreds, and thousands places, which are powers of ten, each place value is a power of two
  - 1101 represents 1 one, 0 twos, 1 four and 1 eight (or 13)



Explain Yourself

- **Exercise 11**: In binary, how do you represent these?
  - (Assume that these are decimal numbers)
  - 3
  - 5
  - 10
  - 8
  - 130
  - 255

Meet

- The first primitive types we'll look a
- **Whole numbers**, including zero a
  - 27 and -345 are integers, 7.25 is
- Theoretically, integers are infinite
  - No matter how big, there's always room for another digit
- Computer-implemented integer primitives are **finite**
  - Stored in fixed area of memory, so different "sizes"
  - Represents "integer concept", limited by constraints
  - Similar to the way B/W photo represents reality

Signed and Unsigned

- Computer integers come in two varieties
  - Signed integers hold both positive and negative numbers
  - Unsigned integers hold only zero and the positive numbers
- For any given amount of storage:
  - Both types can store the same "number of numbers"
    - Eight bits can store 256 numbers whether signed or unsigned
  - The unsigned number represents a greater (2X) magnitude
    - With 16 bits, unsigned = 0 – 65,535 (65,536 values)
    - As an unsigned = -32,768 to 32,767 (65,536 values)
- **All** Java integers are signed

The "Four Families" Integers

- The **byte** uses 8 bits of storage
  - Represents 256 numbers from -128 to 127
- The **short** uses 16 bit of memory
  - Represents 65,536 numbers from -32,768 to 32,767
- The **int** (the "natural sized") uses 32 bits of memory
  - Represents slightly more than 4 billion numbers
  - -2,147,483,648 to 2,147,483,647
- The **long** uses 64 bits of memory
  - Represents numbers from (roughly) +/- nine quintillion

Defining Integer Variables

- Write literal numbers (values) without commas or dots

```
byte   myAge           = 55;
short  milesToGo       = 23455;
int    populationUSA   = 263456732;
long   populationWorld = 5374678328;
```

- Literals use **int** storage, even if small
- Literals assume number is decimal (base 10)
- You can change these assumptions if you like

Initializing long Variables

- This line, from the previous slide, doesn't compile:

```
long populationWorld = 5374678328;
```

  - The literal **5374678328** is too big for 32 bits
- To write a literal integer that uses 64 bits, add the suffix **L** to the end of the number, like this:

```
long populationWorld = 5374678328L;
```

  - Avoid using the lowercase "l" because it looks like a "1"

Number Bases

- Integers are written as decimal, but stored as binary
  - Sometimes, it's easier to write using a different base
  - Suppose you want an applet background to match this:

```
<body style="background-color : #fffbf7;">
```

  - HTML color is a hexadecimal or base-16 number
- To write integers in another base, you add a prefix
  - Literals that start with zero are **octal** (base 8)
  - Literals starting with zero-x are **hexadecimal** (base 16)

```
Color bgColor = new Color(0xfffbf7);
```

Express Yourself

- **Exercise 12**: Write a declaration that defines and initializes this variable. Use smallest type possible.
  - Define and initialize a variable named **students** that holds the number of students in the class right now.

Express Yourself

- **Exercise 13**: Write a declaration that defines **and** initializes this variable. Use **smallest** type possible.
  - Define and initialize a variable that holds a whole number representing the number of inches from the Clark Computing Center in Costa Mesa to the magnetic North Pole of the ex-planet Pluto. (You can guess if you don't know the exact number.) Choose your own name for the variable.

Express Yourself

- **Exercise 14**: Write a declaration that defines **and** initializes this variable. Use **smallest** type possible.
  - Define a variable named **occStudents** that holds the number of students registered at OCC (about 25,000).

Express Yourself

- **Exercise 15**: Write a declaration that defines **and** initializes this variable. Use **smallest** type possible.
  - A variable named **voters** that can hold the number of voters in the next US presidential election. (Make a reasonable guess of the number.)

Express Yourself

- **Exercise 16**: Which are legal Java **integer** **literals**?
  - 3
  - 2+3
  - "2+3"
  - 5,280
  - 0L
  - 0.333333
  - 0xcafebabe
  - 0xCafeBabel
  - 083

Express Yourself

- **Exercise 17**: Write Java statements to:
- Declare two **int** variables, **x** and **y**, initializing **x** to **10**.
- Assign the value **3** to the **int** variable **y**.
- Update the value of **x** by adding **5** to it.
- Create an **int** variable **z**, copy the contents of **y** into it.
- Exchange (swap) the contents of **x** and **y**. Create another variable or two if you need them

Integer Wrap-around

- Integer **wrap-around** or **overflow** when too big

```
(400000 * 400000 / 400000) == 2715?
2147483647 + 1 == - 2147483648?
```

- Which integer type should you use in your programs?
  - Unless you have a reason not to, use the **int**
  - Smaller sizes won't necessarily save memory
  - Calculations always use **int** or **long**

Integer Input

- The integer value 123 in your program is stored like this:
  - 0000-0000 0000-0000 0000-0000 0111-1011
- When user enters 123 at the keyboard, it is **not** a number
  - Instead it is three characters '1', '2', and '3', like this:
    - 0000-0000 0011-0001
      0000-0000 0011-0010
      0000-0000 0011-0011
- Integer input must **convert** from one to the other

Scanner Input

- You can use the **Scanner** class to read **and** convert
  - Create an **int** variable (**miles**) and a **Scanner** object
  - "Prompt" the user for the correct kind of input
  - Use the **Scanner**'s **nextInt()** method to read

```
14    //-----------------------------------------------------------------
15    public static void main (String[] args)
16    {
17       int miles;
18       double gallons, mpg;
19
20
21
22
23
24
```

1 2

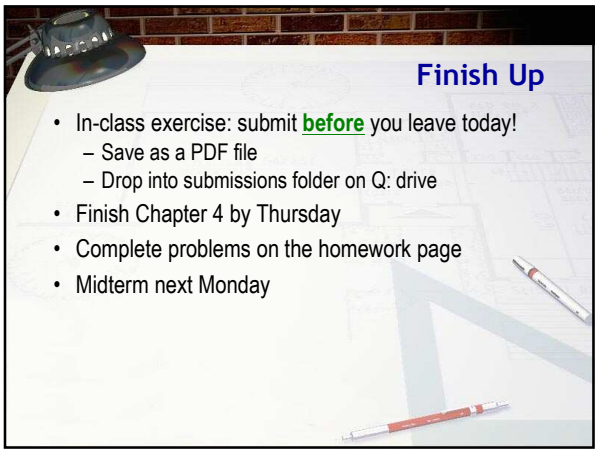
```
C:\WINDOWS\System32\cmd.exe

C:\docs\cs170F05\week03>java GasMileage
Enter the number of miles: 5.25
Exception in thread "main" java.util.InputMismatchException
        at java.util.Scanner.throwFor(Unknown Source)
        at java.util.Scanner.next(Unknown Source)
        at java.util.Scanner.nextInt(Unknown Source)
        at java.util.Scanner.nextInt(Unknown Source)
        at GasMileage.main(GasMileage.java:23)
```

ACM Console Input

- ACM programs use the method named **readInt()**
  - Provide a prompt string when calling the method
  - "Catches" runtime errors and asks the user to re-enter

```
11  import acm.program.*;
12
13  public class ACMGasMileage extends Conso
14  {
15     //-----------------------------------
16     //  Calculates fuel efficiency based o
17     //  user.
18     //-----------------------------------
19     public void run()
20     {
21        int miles = readInt("Enter whole number of miles (25) : ");
22
23        double gallons = readDouble("Enter gallons of gas (3.5) : ");
24
25        double mpg = miles / gallons;
26
27        println ("Miles Per Gallon: " + mpg);
28     }
29  }
```

ACMGasMileage
File Edit
Enter whole number of miles (25) : 50.25
Illegal numeric format
Enter whole number of miles (25) : 50
Enter gallons of gas (3.5) : 2.3
Miles Per Gallon: 21.739130434782612


Finish Up
• In-class exercise: submit before you leave today!
– Save as a PDF file
– Drop into submissions folder on Q: drive
• Finish Chapter 4 by Thursday
• Complete problems on the homework page
• Midterm next Monday